# Understanding Rust and Crustify Research

*A beginner guide to your work with Marius*

Prepared for Shaurya Verma • November 12 2026

This document explains the ideas behind your Marius research from the beginning. You do not need prior Rust or formal verification experience. It connects the language basics to your meeting notes, the new Crustify research, and a small experiment you could propose on your call.

## The recommendation in one paragraph

Start with one real Crustify buffer operation, write down exactly what must remain true about its memory and what C is allowed to do, and try to establish that property with an independent verification tool. Only then add an LLM to help construct the proof. This keeps the problem small enough to learn from and makes the result more meaningful than asking a model whether code looks safe.

## What is established and what is proposed

- Your meeting notes and November 4 chat establish the motivation: combine LLMs with traditional verification, with a later emphasis on modeling hazards and exploring checked proofs.
- The literature survey identifies relevant models, tools, and their stated limits. Numbered references at the end link to primary sources and your earlier discussion.
- The experiment, tool recommendation, and first week plan are proposals. No new verifier installation, Crustify code experiment, completed proof, or paid API experiment is reported here.

## How to use this guide

Read Chapters 1–5 for the foundations. Chapters 6–8 explain what the research is actually trying to do. Chapters 9–11 give the concrete experiment, evaluation, and meeting agenda. The final pages contain a glossary and a reading list.

If the call is imminent, read Chapter 11 first. Its summary is deliberately modest about what has been accomplished and specific about what to ask Marius.

# 1 Why memory safety is a research problem

A program stores data in memory. A pointer identifies where some data lives. An allocation is a region of storage reserved for a value or collection of values. The difficult part is not merely finding an address; it is knowing whether the storage still exists, whether it contains a valid value, and whether this part of the program is allowed to access it now.

## Four failures to recognize

- Out of bounds: reading element 12 of a buffer that only contains 10 elements. A numeric address might exist, but it is outside the storage the operation is allowed to use.
- Use after free: using a pointer after its allocation has been released. The address might now hold unrelated data, or appear to work until the allocator reuses it.
- Double free: releasing the same allocation twice. Two pieces of code may each mistakenly believe they own it.
- Invalid aliasing: using multiple access paths in a way that violates the language's rules. In Rust, these rules can matter even if every address is inside a live allocation.

## Undefined behavior is not just a crash

Undefined behavior, usually shortened to UB, means an execution violates requirements on which the language and its compiler rely. The language no longer promises the ordinary behavior you expect. A crash is one possible result, but silent wrong output or surprising optimization effects are possible too. "It ran correctly on my machine" is therefore weak evidence about the absence of UB. [2]

A normal handled error is different. For example, safe Rust indexing can panic when an index is out of range. A panic is a specified failure mechanism, not automatically undefined behavior. Similarly, leaking memory is undesirable but does not by itself violate Rust's memory-safety guarantee.

## How this relates to your earlier MTE and Scudo work

MTE is hardware memory tagging, and Scudo is a hardened allocator. Those kinds of defenses can detect or mitigate classes of memory misuse at runtime. This new direction asks a different question: can we establish, before or beyond a particular run, that a small program component obeys a precise safety property? Detection, mitigation, and proof can complement each other. None should be described as interchangeable.

## The central question

Can we replace "we tried some inputs and found no problem" with a defensible statement about every execution covered by an explicit model and its assumptions? That last phrase matters: even a formal proof is only as broad as the theorem, model, and trusted assumptions behind it.

# 2 The Rust basics you need first

Rust aims to make memory-safe programming practical without requiring a garbage collector. Its compiler tracks ownership and borrowing. You can think of ownership as responsibility for a value, including when its resources should be released. Borrowing is temporary access without taking over that responsibility. [1]

## Ownership and moving a value

```
let a = String::from("hello");
let b = a;          // Ownership moves to b.
// println!("{a}"); // Rejected: a no longer owns the String.
println!("{b}");
```

A String owns heap storage. Moving it does not mean two independent owners may later free the same buffer. The old binding can no longer be used as its owner. Some simple types, such as integers, implement Copy, so assignment copies them instead. Do not generalize the String example to every type.

## Shared and mutable borrowing

```
let mut values = vec![10, 20, 30];
let first = &values[0]; // Shared reference to an element.
println!("{first}");
values.push(40);        // Fine after that borrow is no longer used.
```

The ampersand creates a reference. An &T reference provides shared access; an &mut T reference provides exclusive access for mutation. Rust normally allows either shared references or an exclusive mutable reference to the same relevant data while those borrows are active. There are carefully controlled interior-mutability mechanisms, but they are not the starting point for our first experiment.

Why care about push? Growing a vector may require moving its elements to a new allocation. A reference into the old allocation must not remain usable after that happens. Rust's borrow checker rejects ordinary safe-code patterns that would create this conflict.

## Lifetimes describe validity relationships

A lifetime is not simply a timer or a variable's lexical scope. It expresses how long a reference can validly be used and how that validity relates to other values. A function must not return a reference to a local allocation that is destroyed when the function returns. The compiler can infer many lifetime relationships; explicit lifetime annotations describe relationships rather than extending an allocation's life.

## Three types you will keep seeing

- Vec<T> is a growable sequence of T values. Its length is the number of initialized elements; its capacity is the space it has reserved.
- A slice, written &[T] or &mut [T], is a borrowed view of consecutive elements. It does not automatically own the allocation.
- A raw pointer, written *const T or *mut T, does not carry the same compiler-enforced borrowing guarantees as a reference. It can exist without being safe to dereference.

# 3 Safe Rust and unsafe implementations

Safe Rust code cannot directly perform certain operations that require the programmer to uphold extra obligations, such as dereferencing an arbitrary raw pointer. Unsafe Rust permits those operations in explicit unsafe contexts. It does not turn off the language's rules or make undefined behavior acceptable. [1, 2]

## A safe interface may contain unsafe code inside

A standard collection can provide a safe method while its implementation uses raw pointers internally. The library author is responsible for making every allowed use of that safe method uphold the required invariants. "Sound abstraction" means that safe callers cannot use the interface, within its stated safe contract, to trigger undefined behavior through an invalid implementation.

That is why your meeting note about a safe reproducer is useful. A small client containing only safe Rust can still expose a bug in a library's unsafe internals. The defect would be in the unsafe abstraction or another trusted component, not evidence that arbitrary correct safe Rust routinely permits memory corruption.

## What forbid unsafe code does and does not show

```
#![forbid(unsafe_code)]

fn main() {
    // Call safe APIs here.
    // The dependency may contain unsafe internals.
}
```

The attribute forbids unsafe code in the scope to which it applies. Putting it in the reproducer crate helps stop a model from "winning" by inserting an obviously unsafe raw-pointer operation into its own test. It does not recursively prove that every dependency is safe, and it does not make a sanitizer omniscient.

A useful reproducer must satisfy several independent checks: the intended restrictions really apply, the client uses the allowed API, the relevant sanitizer or interpreter runs, and the reported failure is relevant to the property being investigated. Compiler success alone is not that evidence.

## The LLM harness is a search procedure

Your note proposes asking the LLM to find a compilable combination of Rust statements that triggers a checker. That is a bug-finding strategy. A successful, validated counterexample can be valuable. An unsuccessful search only says that this search did not find a counterexample with its time, tools, and inputs. It does not prove that no counterexample exists.

This is the meaning of the "lower bound" intuition: confirmed failures show at least those failures exist; the number found does not measure every latent bug. A stronger method must reason about executions the model never explicitly wrote or ran.

## A practical warning about reward hacking

The model could weaken the test, change the target, suppress the checker, or use an irrelevant crash. Restricting unsafe code is one guardrail, not a complete defense. Keep the target and acceptance checks outside the model's writable scope and validate its outputs independently.

# 4 The Rust and C boundary

FFI means foreign function interface: the mechanism that lets code in one language call code in another. Crustify's setting involves Rust and C. A Rust wrapper may call C to allocate, fill, resize, or free a buffer. The Rust compiler cannot infer all the facts about an arbitrary C implementation that it normally tracks for ordinary safe Rust.

## The wrapper needs more than a function signature

Suppose C receives a pointer and a length. Will it only read those bytes? Can it modify them? Will it save the pointer for later? Can it free or reallocate the buffer? Can it call back into Rust while the operation is in progress? These behaviors can change whether a Rust reference is valid. The argument types alone often do not express enough.

An effect contract records those answers. It describes the permitted effects and the state guaranteed on return. A Rust-side proof can assume that contract, but a separate argument is needed to show that the real C implementation obeys it. Otherwise the result is explicitly conditional.

## A reallocation example

```
Before C call:  ptr A --> allocation A
Rust view:     &data --> allocation A

C may resize:  ptr B --> allocation B
               allocation A may be freed

Danger:        old Rust view still refers to A
```

The bug is not that C changed a number. The bug is using a view whose storage or access permissions are no longer valid. Reallocation can sometimes keep the same numeric address, but a sound argument cannot rely on that accidental outcome. Allocation identity and provenance matter as well as the address.

## Three separate obligations

- Storage: the region must exist, have the right extent and alignment, and stay alive for the access.
- Contents: the accessed elements must be initialized and valid for their type.
- Permissions: the access must respect the applicable borrowing and aliasing rules, including effects of other access paths.

A proof of one does not automatically prove the other two. For example, an in-bounds write through C may still conflict with a live shared Rust reference. Conversely, changing an unrelated C-side metadata field is not automatically an aliasing violation.

## Why this is a good research target

The boundary concentrates assumptions that are easy to overlook in code review. If we can represent those assumptions precisely for a small reusable buffer abstraction, we may help many translated callers without having to prove an entire mixed-language application at once.

# 5 What Crustify does

Crustify is the translation research system discussed with Marius. The November 7 2026 preprint describes translating C into Rust while managing ownership and the remaining unsafe boundary. Its relevance here is not that translation eliminates the need for verification. It is that the generated code and reusable primitives provide concrete targets for it. [4]

## The three components relevant to your project

- ffibox provides ownership and borrowing primitives intended to avoid creating Rust references into memory that C may still alias. The exact guarantee depends on the specific primitive and how callers use it.
- Wavefront handles dependency ordering and uses agent-authored ownership information during the translation process. An ownership record is useful guidance, but agreement with a generated record is not itself proof that the record is correct.
- ffimap measures parts of the unsafe surface. The paper explicitly describes it as a linter rather than a sound and complete verifier. A lower count of risky constructs is not a theorem that the remaining program has no UB.

## Specific gaps reported by the paper

The paper reports that ffimap misses some implicit references formed by autoref through raw pointers and references involving some unmapped -sys structures. Autoref means Rust inserts a reference implicitly for an operation, so looking only for an explicit ampersand can miss it. A -sys crate commonly contains low-level foreign-library bindings.

The paper also identifies unsupported ffibox cases, including owning-element vectors and arrays that C grows through a borrowed pointer and count. These are useful leads to discuss, not evidence that every existing ffibox use is broken. We have not reproduced these cases or inspected every artifact repository in this research pass.

## Two different kinds of contribution

An engineering contribution could improve a documented blind spot in ffimap or extend a primitive. A verification contribution could establish a precisely stated property of an existing primitive and develop a repeatable checked-proof workflow. Both could help Crustify, but they answer different questions and need different evaluation.

## My recommendation for your call

Ask Marius for one actual method, the current branch, and a realistic allowed C behavior. The best first target is small enough that a human can understand its contract and that a deliberately broken version can be constructed. Avoid committing to “prove Crustify safe” before you know the representation, supported language features, and proof infrastructure.

The narrow initial claim might be: this resize wrapper preserves an initialized-buffer invariant under a stated allocation contract. That is much more actionable than “Rust code can never have UB,” and it makes the remaining assumptions visible.

# 6 Your meeting notes explained and clarified

Your notes describe two avenues: extending static analysis across Rust and C, and using formal verification with LLM-assisted proofs. Your later November 4 discussion emphasizes first studying verified primitives, modeling the hazards, and choosing a plan before API-key experiments. These are related directions, not evidence that both full systems have already been designed. [3]

## Avenue one is static analysis

Static analysis examines a program without running each possible execution. An analysis might track which allocation a pointer refers to, which calls can free it, and whether a later access can use it after its lifetime ends. To scale, it usually merges many possible runtime states into an abstract representation.

Imagine a branch where only one path frees a pointer. If the analyzer cannot establish which branch will be taken, it may conservatively retain both possibilities. A later read becomes a warning. If that branch is impossible for real inputs, the warning is a false positive. This is a common cost of overapproximating possible behavior.

Your notes use “complete and not sound” for “no false negatives but false positives.” Terminology varies with what the procedure is deciding, so avoid using that label alone. For a may-bug analysis intended to cover all modeled executions, the usual sound-overapproximation goal is no missed modeled behavior, accepting possible false alarms. Concrete tools can still miss bugs because of unsupported features, unsound shortcuts, or inadequate models.

An LLM could propose tests or explain why a warning seems infeasible. But its opinion is not a proof that a warning is false. A robust workflow keeps an unresolved category and requires independently checkable evidence before dismissing a warning.

## Avenue two is formal verification

Formal verification states a property mathematically and checks a proof against a model. The hard part includes expressing the real program and its memory behavior accurately, not just writing the final proof. Lean, Rocq, and Isabelle are proof-assistant ecosystems, not magic importers that automatically understand arbitrary Rust and C.

An LLM can propose proof steps, helper lemmas, annotations, or candidate contracts. A trusted checker can reject an invalid proof step. But it cannot rescue the wrong theorem: if the theorem assumes away the bug or models the wrong program, a valid proof can still be irrelevant to the real system.

## How the two avenues fit together

Static analysis can find candidate obligations or select a suspicious method. Formal verification can establish a deeper property for a restricted target. Tests and dynamic checkers can expose mistakes in implementations or models. The research need not discard testing; it should state exactly what each type of evidence establishes.

# 7 What a proof would actually look like

A useful specification has preconditions, postconditions, and an invariant. Preconditions are facts required before an operation. Postconditions describe what it guarantees afterward. An invariant is a fact that must remain true at the boundaries where the abstraction promises it, often before and after every public operation.

## A small initialized buffer invariant

For a byte buffer, let p identify storage, n be its initialized length, and c be its capacity. A first invariant could require 0 ≤ n ≤ c; a live allocation large enough for c bytes; valid initialized contents in the prefix [0, n); correct ownership for destruction; and no outstanding access that conflicts with the operation being performed.

```
capacity c = 8
index       0 1 2 3 4 5 6 7
contents    A B C D ? ? ? ?
length n = 4

[0, n) is readable data
[n, c) is reserved space, not necessarily readable data
```

If a read receives an index i with 0 ≤ i < n, the invariant tells us the element exists and is initialized. The index check alone would not establish that. Constructors must create the invariant, mutations must preserve it, and destruction must consume ownership correctly.

## An illustrative theorem rather than a finished proof

“For every state satisfying BufferInvariant, and every index below its length, read returns the corresponding stored byte without violating the modeled memory rules, provided the foreign code obeys the stated effect contract.” This sentence makes the quantification and assumptions visible. A real theorem would also name the program semantics and treatment of failures.

The first model can use bytes to avoid extra complications of arbitrary T values, destructors, and type validity. Restricting the first target is legitimate if the restriction is explicit. It is not legitimate to later describe the result as covering every Rust type or arbitrary C callbacks.

## Why kernel checking is useful but not enough

A proof assistant’s kernel checks that a proof follows its logical rules. This can make an LLM’s confidence irrelevant: invalid proof steps fail. However, the total trusted chain still includes the statement, imported axioms, formal semantics, and whatever translation connects the real source to the model. Those assumptions must be documented.

## Vacuous success is a real failure mode

If the precondition accidentally says n < 0 and n ≥ 0, there are no allowed states. A theorem can then hold without telling us anything about real buffers. Similarly, assuming “this pointer is always valid” may hide precisely the property we hoped to establish. Test that ordinary intended states satisfy the preconditions, and review assumptions separately from proof success.

# 8 Choosing an initial verification approach

## RefinedRust and Rocq are the closest buffer starting point

RustBelt models ownership and sharing logically to justify safe interfaces implemented with unsafe code. RefinedRust builds a more automated verification approach and includes a simplified Rustonomicon vector example. Its representation separates allocation ownership from initialized contents and spare capacity. That is unusually relevant to the buffer question here. [5, 6]

Important limits remain. The example is not a proof of the entire production standard-library vector. The published scope excludes a complete operational aliasing model and some validity and pointer-cast details. A successful proof in that model must not be advertised as full Tree Borrows compliance. A 2026 extension improves practical support, but frontend correspondence and parts of the semantic infrastructure remain trusted. [7]

## Lean is useful for an explicit small model

A hand-written Lean model could represent allocation identities, initialized ranges, and allowed operations clearly. It may be the quickest way to learn the invariant and exercise a proof-generation loop. But a proof about that model does not automatically verify the actual ffibox source. Keep the model-to-source correspondence as a separate obligation.

The new lean-miri project is worth a feasibility look, not an assumption of mature support. Its own status distinguishes importing MIR, executing code, and proving a safety theorem. A successful process exit is not enough to establish completed guest execution or universal correctness. [17]

## Isabelle and other systems

AutoCorrode is relevant separation-logic infrastructure with a restricted Rust-oriented frontend. It is not an arbitrary Rust/C verifier. Aeneas is useful related work for safe Rust, but its stated current scope should not be confused with automatic verification of arbitrary unsafe ffibox internals. Choose infrastructure partly based on what Marius and the group already know. [8, 9]

## Keep operational aliasing as a separate decision

If the main theorem is about aliasing permissions, choose and pin the aliasing model. Tree Borrows provides a specific model and formalization. A recent separation-logic proposal for unsafe Rust is work in progress rather than a completed general verifier. Do not call a proof about length arithmetic an aliasing proof. [15, 16]

## My recommendation

First explore whether a small real primitive fits the existing RefinedRust buffer approach. If setup or unsupported features dominate, use a small explicit model to learn the proof obligations, while clearly labeling the source-correspondence gap. Do not begin by building a whole Rust and C translator. Ask Marius to choose whether the first milestone prioritizes a real-source theorem or a clear proof-of-concept model.

# 9 Related tools and the novelty question

## Dynamic and static baselines

Miri interprets Rust and can catch many invalid executions. Its experimental native-library support has important tracking limitations, so passing it is not a certificate for arbitrary native C behavior. MiriLLI combines Rust and LLVM interpretation for mixed-language executions; it is a useful dynamic baseline, not proof over all executions. [10, 11]

Crema is an early-stage static analyzer targeting cross-language heap-management issues such as leaks, use after free, and double free. Its stated scope does not equal complete Rust reference-aliasing verification. FFIChecker is another related cross-language memory-management analyzer with an older pinned toolchain. Compatibility work may be necessary before either is usable on current artifacts. [12, 13]

Kani can check a bounded or otherwise explicitly modeled Rust harness and supports foreign-function stubs. A stub is a replacement model of C behavior. A checked result applies to that replacement unless a separate argument connects it to real C. Record bounds and assumptions instead of reporting simply “proved safe.” [14]

## LLM assisted proof construction already exists

Aria reports agent-assisted reconstruction of RustBelt and Iris proofs checked by a proof system. KaPilot studies generating specifications for unsafe Rust with Kani. These make “we used an LLM to write Rust proofs” insufficient as a novelty claim. Their reported results also do not imply that arbitrary new mixed Rust/C programs are solved. [18, 19]

The promising question is more specific: can reviewed C-effect contracts and reusable buffer invariants help an LLM construct checked proofs for actual Crustify primitives, while detecting inadequate specifications and invalid implementation changes? The contribution could be in the modeling, automation, real target coverage, or evaluation. That novelty still needs confirmation with Marius and a deeper related-work review.

## Other leads worth keeping in the hub

BorrowSanitizer targets aliasing detection in mixed-language settings, and SafeFFI studies practical sanitizer placement at boundaries into safe pointer types. They are important comparisons if the project shifts toward runtime instrumentation. Neither should be presented as interchangeable with a universal proof of the proposed wrapper. [20, 21]

## The standard for a useful result

It is more convincing to verify one real property, expose the assumptions, and show that meaningful broken variants fail than to generate many persuasive-looking proofs of weak statements. Tool failure and proof failure must remain distinct from demonstrated bugs. Unsupported targets are valuable information about feasibility, not something to silently remove from the denominator.

# 10 The proposed first experiment

This is a plan, not an experiment already performed. Start with one byte-buffer operation and exclude concurrency, callbacks, arbitrary pointer-integer casts, and complex element destructors unless the chosen target requires them. If those features are central, choose a different first target or explicitly budget the modeling work.

## Step one is to freeze the target and claim

Get the exact ffibox method, repository version, and expected C behavior from Marius. Write the intended property in plain English before choosing proof syntax. Decide whether the operation must preserve bounds, initialized contents, ownership, or operational aliasing. List the properties intentionally left outside the first theorem.

## Step two is to reproduce an existing checked example

Build a tiny example in the selected framework and rerun its original checks. Record tool versions and commands. This tests whether the infrastructure works before asking an LLM to solve an unfamiliar target. Then write or adapt a reviewed specification for the real method.

## Step three is to use correct and broken pairs

- Correct resize versus a version that keeps using an old pointer after a successful moving reallocation.
- Correct failure handling versus a version that loses or incorrectly frees the original allocation on resize failure. Use the actual allocator contract, not an assumed generic realloc behavior.
- Correct initialized-length update versus a version that claims uninitialized spare capacity is valid readable data.
- Correct ownership release versus a version that permits two owners to free the same allocation.

Not every framework can express every mutant. Report that limitation. A failure to verify a broken program is not automatically a counterexample; the tool may simply be incomplete or time out. Where possible, obtain a concrete failing execution or a precise violated obligation.

## Step four is to add the LLM only to proof construction

Keep the program, theorem, contracts, imports, and acceptance policy fixed. Let the model propose proof steps or approved helper lemmas. Reject attempts to add unchecked assumptions, use proof escape hatches, change the target, or relax preconditions. Check the result in a fresh environment so stale build artifacts cannot masquerade as success.

## Step five is to compare fairly

Compare baseline automation, a one-shot model attempt, and a model with verifier feedback under equal budgets. Measure accepted proofs, human specification effort, time, tokens or cost, unsupported cases, and invalid-edit attempts. Review which theorem was actually checked and what assumptions it depends on. This is the beginning of an evaluation, not a promise of publication-ready results in a week.

# 11 Your call agenda and first week milestones

## A short explanation you can say aloud

“I looked at how verified Rust buffers separate allocation ownership from initialized contents. A bounds check alone is not enough unless the metadata accurately describes valid memory. I would like to start with one ffibox primitive and an explicit contract for what C can do, then prove one property. After that, we could let an LLM help construct the proof while an independent checker validates it. I need your advice on the target, the property, and the framework.”

## Questions to ask Marius

- Which exact ffibox primitive and branch should I study first? Is there a small method with a known tricky safety condition?
- Is the first target bounds and initialization, ownership, or operational aliasing? If aliasing, which model and version should we use?
- Should I assume a C contract for the first result, or do you want the C behavior verified as part of it?
- Does the group already use Rocq, Lean, Isabelle, or another framework? Are there proofs or harnesses I can reproduce?
- What would count as a useful first contribution beyond existing LLM proof and unsafe-Rust specification work?
- Would an ffimap blind-spot fix be useful in parallel, or would that distract from the proof-oriented goal?

## A realistic sequence of milestones

First, identify a target and write the intended theorem and assumptions. Second, reproduce one existing proof and confirm tool support. Third, specify one real operation and build meaningful correct/broken examples. Fourth, introduce a restricted proof-generation loop. Finally, compare approaches and document limitations. These are ordered milestones, not guaranteed one-day tasks.

## What you can honestly say has been done

The recent discussion has been recovered, the research direction clarified, primary literature and tool limitations surveyed, and a concrete starting plan proposed. No new checked proof of ffibox has been produced in this pass. The first useful decision for the call is what precise theorem to attempt, not which model API key to buy.

## What to write down before the meeting ends

Record the selected method and commit, the desired property, the allowed C effects, the preferred framework, and the next observable deliverable. If the choice remains open, record the two options and the experiment that would distinguish them. This will keep the next week from turning into an unbounded tool survey.

# 12 Glossary and self check

**Allocation.** A reserved region of memory. It has an identity and lifetime, not just an address.

**Aliasing.** Multiple ways to reach the same memory. Whether particular accesses are allowed depends on language rules and permissions.

**Borrow.** Access to a value without taking ownership. Shared and exclusive borrowing have different rules.

**Contract.** An explicit statement of required inputs or state and permitted effects or guaranteed results.

**Counterexample.** A concrete execution or model state that violates a claimed property.

**Invariant.** A property maintained at the relevant boundaries of an abstraction or loop.

**Lemma.** A helper theorem used to prove another theorem.

**MIR and LLVM IR.** Intermediate program representations used by compiler and analysis tools. Importing either does not alone prove correspondence to all source behavior.

**Provenance.** Information about a pointer's origin or allocation relationship that can matter beyond its numeric address.

**Reproducer.** A small program that reliably demonstrates a particular observed problem.

**Sanitizer.** Instrumentation or a runtime checker that detects certain invalid behaviors on executions it observes.

**Trusted computing base.** The components and assumptions the result relies on rather than independently proves.

**Vacuous proof.** A logically valid result that says little about intended behavior, for example because its assumptions permit no real case.

## Check whether the main ideas make sense

Why is index < len insufficient by itself? Because the allocation and initialized-length metadata must be valid, and access permissions must hold. Why is no sanitizer failure insufficient? Because only particular executions and checks were exercised. Why can a checked proof still miss the point? Because it may prove the wrong statement, use unrealistic assumptions, or lack a justified connection to the real source.

What is the first research deliverable? A small, explicit target and property with a reproducible checking procedure, not a blanket assurance that generated Rust is safe.

# 13 Sources and suggested reading order

Start with sources 1–4, then read the relevant parts of 5–7. The rest are comparisons and feasibility leads, not prerequisites for your first conversation. Research papers and repository status pages can change; pin versions when beginning an experiment. Sources were identified in the November 12 research pass.

**1 Rust Book ownership and unsafe Rust** https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html

**1b Rust Book unsafe Rust** https://doc.rust-lang.org/book/ch20-01-unsafe-rust.html

**2 Rust Reference undefined behavior** https://doc.rust-lang.org/reference/behavior-considered-undefined.html

**2b Requirements of slice from raw parts** https://doc.rust-lang.org/std/slice/fn.from_raw_parts.html

**3 Your November 4 discussion** https://chatgpt.com/c/6aeaf053-ace8-83e8-9a49-ca7ad63f9a9e

**4 Crustify preprint November 2026** https://arxiv.org/html/2611.10394v1

**5 RustBelt POPL 2018 paper** https://plv.mpi-sws.org/rustbelt/popl18/paper.pdf

**6 RefinedRust PLDI 2024 paper** https://plv.mpi-sws.org/refinedrust/paper-refinedrust.pdf

**7 RefinedRust extension 2026** https://iris-project.org/pdfs/2026-oopsla-refinedrust2.pdf

**8 AutoCorrode repository** https://github.com/awslabs/AutoCorrode

**9 Aeneas repository** https://github.com/AeneasVerif/aeneas

**10 Miri documentation** https://github.com/rust-lang/miri

# 14 Additional research sources

**11 MiriLLI repository** https://github.com/icmccorm/mirilli

**11b Mixed language analysis study** https://arxiv.org/abs/2404.11671

**12 Crema repository** https://github.com/AFx3/crema-static-analyzer

**13 FFIChecker repository** https://github.com/lizhuohua/rust-ffi-checker

**14 Kani stubbing documentation**
https://model-checking.github.io/kani/reference/experimental/stubbing.html

**15 Tree Borrows project and formalization** https://plf.inf.ethz.ch/research/pldi25-tree-borrows.html

**16 Unsafe Rust aliasing verification proposal** https://arxiv.org/html/2603.28326v1

**17 lean miri experimental project** https://github.com/proofs-rs/lean-miri

**18 Aria LLM proof reconstruction** https://arxiv.org/html/2607.06341v1

**19 KaPilot unsafe Rust specification generation** https://arxiv.org/html/2607.21957v1

**20 BorrowSanitizer project** https://borrowsanitizer.com/

**20b Rust BorrowSanitizer goal** https://goals.rust-lang.org/2026/borrowsanitizer.html

**21 SafeFFI paper** https://www.plai.ifi.lmu.de/publications/usenixsecurity26-safeffi.pdf

Source boundaries: your supplied meeting notes are historical notes, not experimentally validated claims. This guide preserves their motivation while correcting terminology and separating conditional guarantees from universal claims. Tool descriptions reflect the cited materials; no compatibility or performance benchmark was run for this guide.